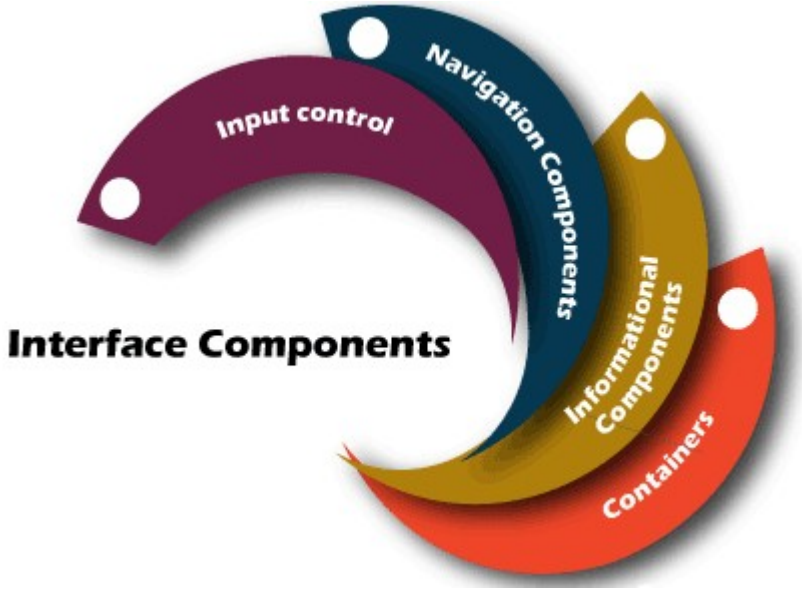


Programska arhitektura uporabniških vmesnikov

Vsebina poglavja

- vrste uporabniških vmesnikov
- strukturna razlika uporabniških vmesnikov
- vrste vhodnih dogodkov
- okenski sistem X
- okenski sistem Wayland
- hierarhija orodij za načrtovanje uporabniških vmesnikov
- orodja za načrtovanje uporabniških vmesnikov
- obravnava dogodkov, vrsta dogodkov, zanka dogodkov
- arhitektura model – pogled - nadzornik

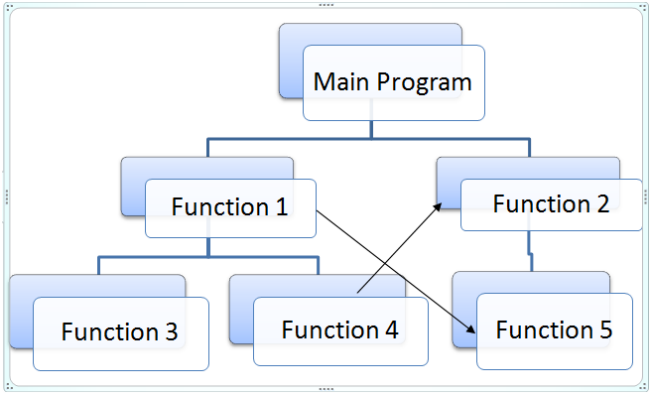


Vrste uporabniških vmesnikov (UV)

- konzolno usmerjeni uporabniški vmesniki
  Akcija→Objekt
  - navadni konzolni vmesniki
  - konzolni interaktivni vmesniki
- grafični uporabniški vmesniki
  Objekt→Akcija
  - preprosti dogodkovni interaktivni vmesniki (X11)
  - dogodkovni interaktivni vmesniki z grafičnim uporabniškim vmesnikom → pisanje tiste kode, ki se izvaja kot odziv na dogodek
    - GTK+, Qt, Motif: odzivne funkcije
    - Java: poslušalci

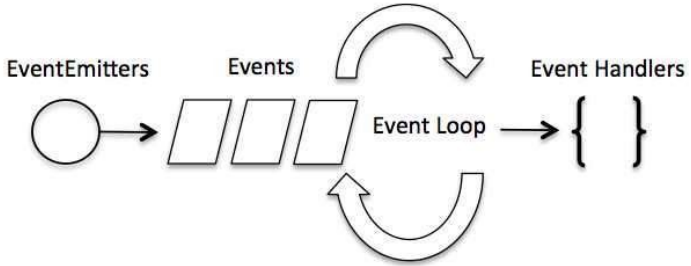
Strukturna razlika uporabniških vmesnikov

- konzolni vmesniki: programer določi, kdaj bo prišlo do vnosa podatkov, izpisa rezultatov, … → izvajanje programa je vnaprej določeno
  postopkovno programiranje



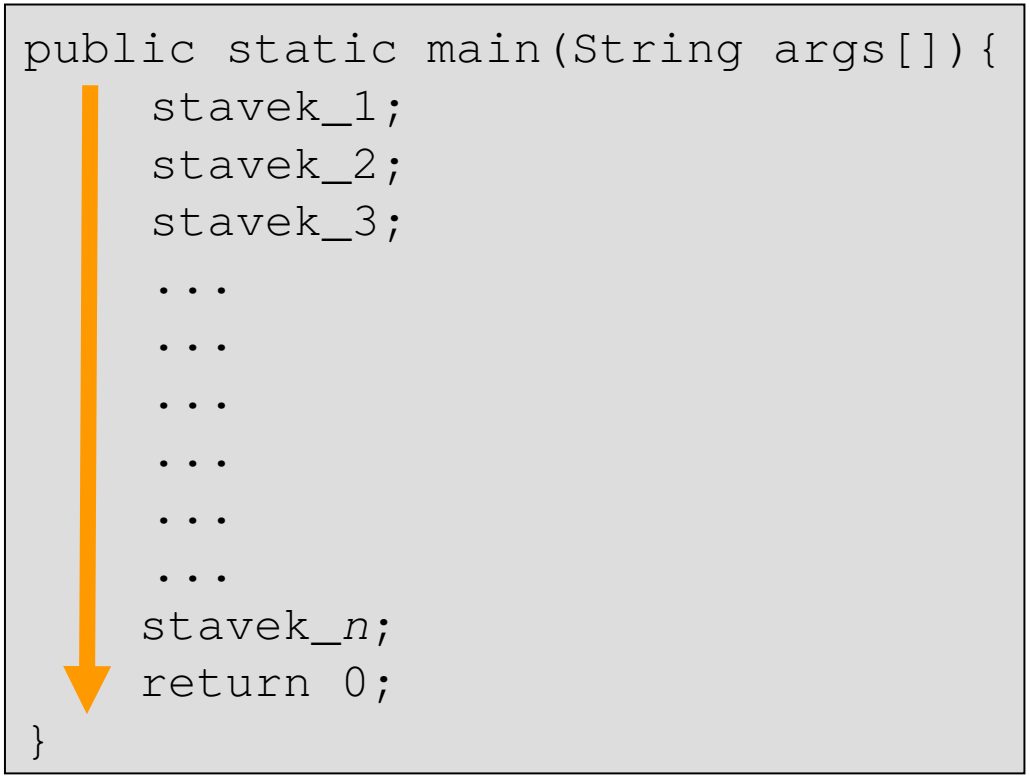
- grafični uporabniški vmesniki: programer implementira uporabniški vmesnik in določi odzive na uporabnikove akcije → uporabnik izvede akcijo, kadar želi (npr. vnos podatkov), sledi ustrezen odziv vmesnika na akcijo (npr. izpis rezultatov, prikaz okna, …)

  dogodkovno usmerjeno programiranje



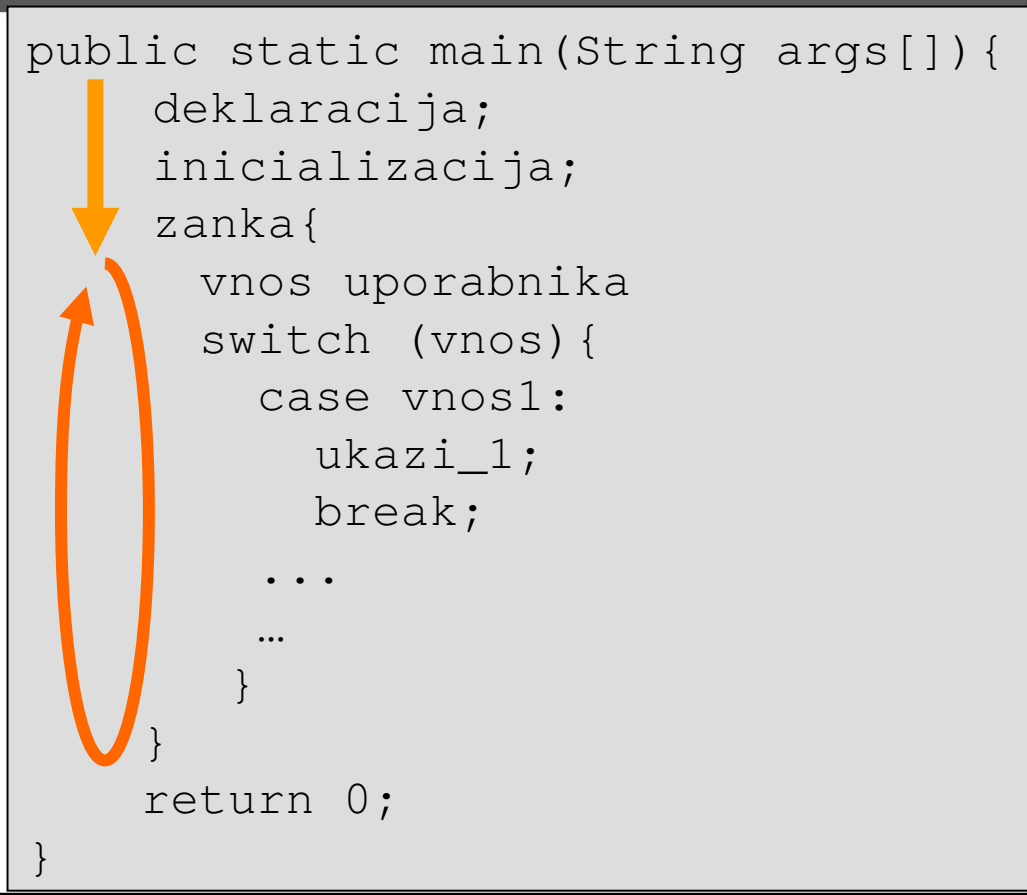
Navadni konzolni vmesniki

- ne omogočajo neposredne interakcije
- zaporedno izvajanje kode od začetka do konca
- potek izvajanja je določen vnaprej
- možno je vplivanje preko vhodnih parametrov



Konzolni interaktivni vmesniki

- omogočajo neposredno interakcijo
- nelinearno izvajanje kode
- vmesnik čaka na vnos uporabnika, po vnosu nadaljuje izvajanje
- potek izvajanja je odvisen od uporabniškega vnosa
- veliko časa neizvajanja



Preprosti dogodkovni interaktivni vmesniki

- omogočajo neposredno interakcijo
- nelinearno izvajanje kode
- potek izvajanja je odvisen od uporabniškega vnosa
- veliko časa neizvajanja
- uporabnik lahko v vsakem trenutku izvaja različne stvari:
  pritisk na gumb, tipkanje, ...
- tem dejanjem rečemo **dogodki**
- na dogodke čaka aplikacija v zanki dogodkov, za katero poskrbi sama



```
public static main(String argv[]){
    deklaracija;
    inicializacija;
    zanka{
      dogodek uporabnika
      switch (dogodek){
        case vnos1:
          ukazi_1;
          break;
        ...
        …
      }
    }
    return 0;
}
```

Vrste vhodnih dogodkov

- uporabniške dogodke lahko ločimo v dve skupini:
  - preprosti vhodni dogodki
    - premik miške
    - pritisk ali spust gumba miške (angl. button p*ress/release*)
    - pritisk ali spust tipke na tipkovnici (angl. *key press/release*)
  - prevedeni vhodni dogodki (visokonivojski dogodki)
    - klik (angl. *click*) ali dvojni klik (dvoklik) miške
    - vstop/izstop miške v/iz komponente (angl. *enter*, *exit*)
    - pridobitev/izguba fokusa (angl. *focus gained/lost*)
    - odtipkan znak (angl. *typed*) → če se lahko generira veljaven znak

Okenski sistem X (X11)

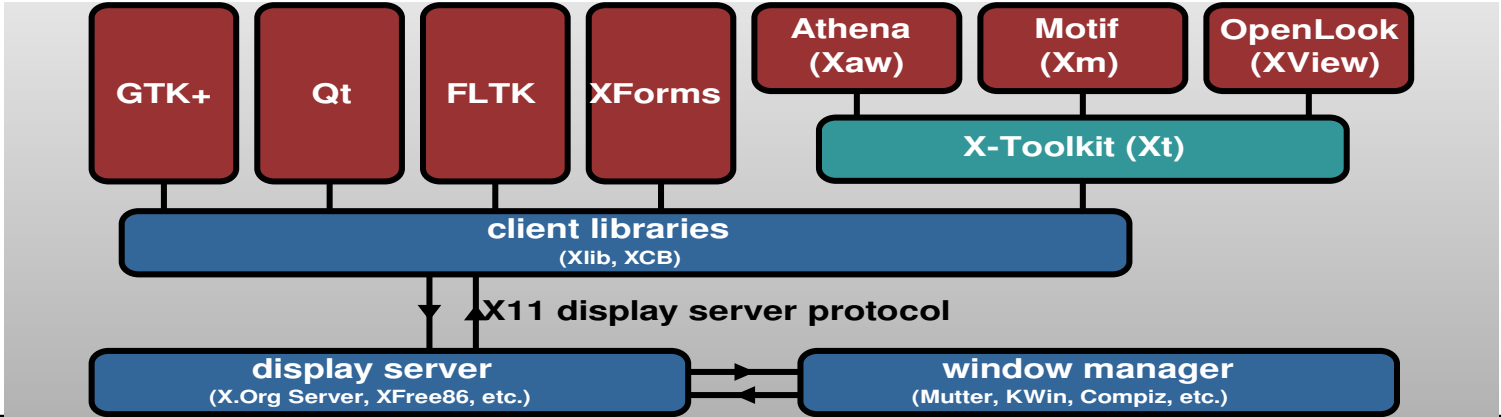
- zahteve:
  - povezati množico **grafičnih** delovnih postaj in jih uporabiti kot učne pripomočke
  - poganjati grafične aplikacije na poljubnem sistemu
  - poganjati grafične aplikacije na oddaljenih sistemih
- rezultat:
  - okenski sistem X, prvo strojno in pozicijsko neodvisno okolje
- lastnosti:
  - neodvisen od operacijskega sistema
  - grafično okolje veliko sistemov (HPUX, AIX, Solaris, …)
  - sistem okna
  - koncept odjemalec/strežnik
  - odjemalec in strežnik lahko na različnih računalnikih

Okenski sistem

- X11 relativno star, se redko(!?) uporablja, vendar ne zahteva strojnega pospeševanja, zato se dobro obnese v navideznih računalnikih
- obstajajo alternative za različne operacijske sisteme
  - Wayland (nadomestilo za X11): Linux, FreeBSD
  - Mir (nadomestilo za X11): trenutno kot kompozitor za Wayland, noben grafični OS ga privzeto ne uporablja
  - Desktop Window Manager: Windows okenski sistem od Windows Vista dalje
  - Quartz Compositor: Mac OS X
  - zgodovinsko pa so še: SunView, HP Windows, Next DPS, ReactOS Explorer, npr. Dojo (okenski sistem za splet, dejansko pa razvojno ogrodje za splet), …
- v glavnem pa se za poganjanje oddaljenih aplikacij uporablja oddaljeno namizje, kar pa bolj obremenjuje prenos

Orodja (*toolkit*) za načrtovanje uporabniških vmesnikov

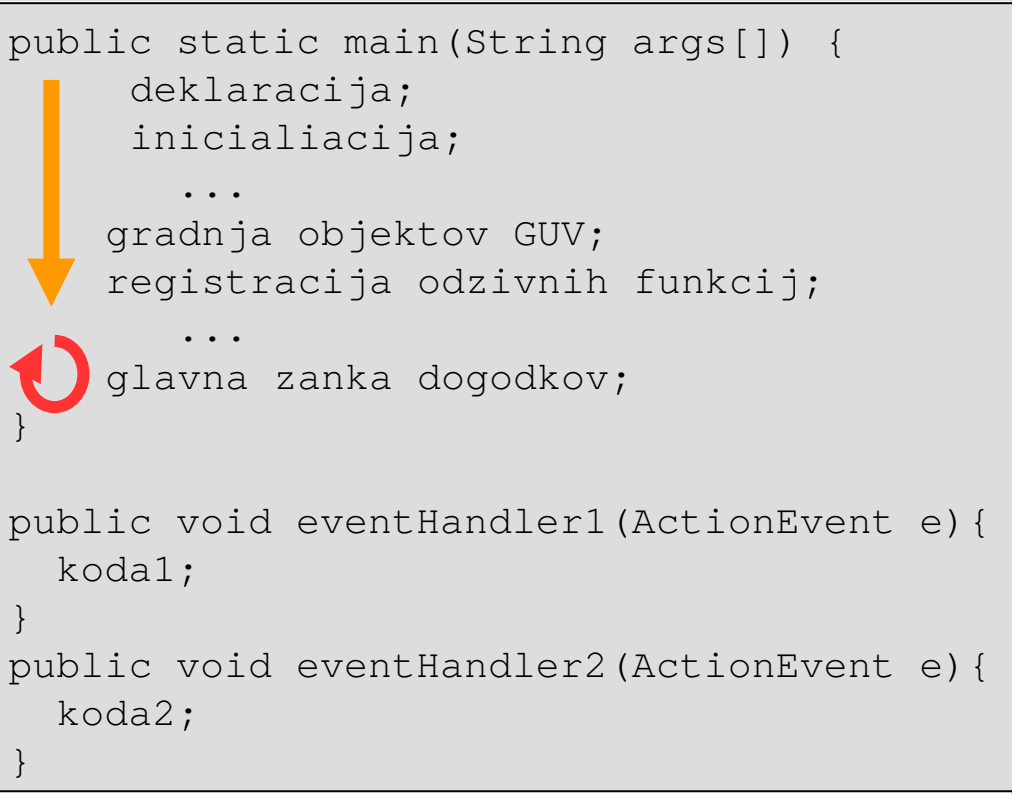
- nekatera programska okolja oziroma jeziki nudijo možnost razvoja dogodkovnih interaktivnih vmesnikov z grafičnim uporabniškim vmesnikom (GUV) z uporabo protokola X11; obstajajo večplatformna in domorodna orodja
  - GTK+, Qt, wxWidgets
  - MFC, Cocoa, Motif
  - Swing, AWT, SWT (IBM/eclipse), JavaFX (nadgradnja GTK+/Qt/...)



Dogodkovni interaktivni vmesniki z grafičnim upor. vmes.

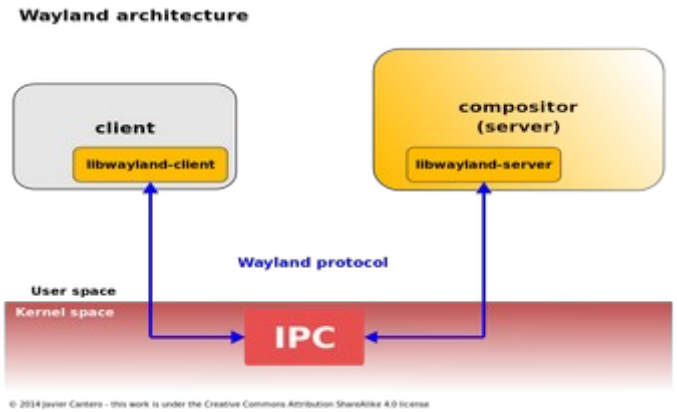
UL FRI

- omogočajo neposredno interakcijo
- nelinearno izvajanje kode
- potek izvajanja je odvisen od vnosa
- veliko časa neizvajanja
- uporabnik lahko v vsakem trenutku dela različne stvari: pritisk gumba, tipkanje, ...
- tem dejanjem rečemo **dogodki**
- interna zanka dogodkov
- potrebna je registracija rokovalnikov dogodkov
- na dogodke čakajo rokovalniki dogodkov - odzivne funkcije



Wayland sitem (protokol)

- je protokol, ki določa komunikacijo med strežnikom (zaslon) in odjemalci, prav tako pa je C knjižnica, ki implementira ta protokol
  - sledi modelu strežnik-odjemalce
  - sestavljen iz dveh nivojev
    - nizkonivojski skrbi za medprocesno komunikacijo
    - visokonivojski rokuje z informacijo, ki jo morate odjemalec in kompozitor izmenjati za implementacijo funkcionalnosti okenskega sistema



  - visokonivojski generiran avtomatsko iz opisov elementov protokola v XML dat.
  - osnovni elementi: **wl_display**, **wl_registry**, **wl_compositor**, **wl_surface**, **wl_buffer**, **wl_output**, **wl_pointer**, **wl_keyboard**, **wl_touch**, **wl_seat**

Značilnosti Wayland-a

- arhitektura: prikazovalnik in kompozitor sta združena (v X11 je kompozitor dodatek)
- kompozitino sestavljanje slike: obvezen v Wayland (opcijski v X11)
- upodabljanje: v Waylandu se upodabljanja (risanje) delegirajo odjemalcem, tudi v primeru pisav in podob (X11 lahko sam poskrbi za upodabljanje)
- varnost: Wayland loči I/O vsakega okna, s čimer zagotavlja zaupnost, integriteto in dostopnost (X11 nima varnostnih funkcij)
- medprocesna komunikacija: Wayland tega ne zagotavlja na nivoju protokola, če je potrebno, mora to zagotoviti namizni sistem (X11 strežnik zagotavlja osnovne metode, uporablja za zagotavljanje funkcionalnosti: X seja, drag-drop, …),
- omrežje: Wayland ne zagotavlja omrežne transparentnost, kompozitor lahko implementira katerikoli oddaljeni protokol (X11 zasnovan da deluje preko omrežja)

Wayland sistem → primeri

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <assert.h>
#include <stdbool.h>
#include <syscall.h>
#include <unistd.h>
#include <sys/mman.h>
#include <xkbcommon/xkbcommon.h>

#include <cairo/cairo.h>
#include <linux/input-event-codes.h>

#include <math.h>
#include <wayland-client.h>
#include <wayland-cursor.h>
#include "xdg-shell.h"
```

Wayland je zamenjava za X11 in je kot tak bistveno bolj raznolik.

```c
struct client_state {
  struct wl_compositor *compositor; // objekt, ki izvaja sestavljanje slik
  struct wl_shm *shm;               // objekt za upravljanje s spominom
  struct wl_seat *seat;             // objekt za vhodne naprave
  struct wl_pointer *pointer;       // objekt za miško
  struct wl_keyboard *keyboard;     // objekt za tipkovnico
  struct wl_display *display;       // objekt, ki skrbi za prikaz
  struct wl_surface *surface;       // površina za risanje
  cairo_surface_t *csurface;        // površina preko katere rišemo na surface
  struct xdg_wm_base *xdg_wm_base;  // razširitev za wailand lupino
  struct xdg_toplevel *xdg_toplevel; // razširitev, ki omogoča okna, popupe
  struct xdg_surface *xdg_surface; // razširitev za površino
  struct xkb_context *xkb_context;  // kontekst za tipkovnico
  struct xkb_state *xkb_state;      // stanje tipkovnice
  struct xkb_keymap *keymap;        // mapiranje tipk na tipkovnici
  struct wl_cursor_theme *ptr_theme;// tema za kazalec miške
  struct wl_cursor *ptr_default;    // izbrani kurzor za prikaz
  struct wl_surface *ptr_surface;   // površina za prikaz kurzorja
};
struct client_state state = { 0 };
```

Wayland sistem → primeri

```
static const char *ptr_names[] = { // imena nekaterih
kazalcev
  "bd_double_arrow",
  "bottom_left_corner",
  "tcross",
};
static const int totPtrs=20;

struct wl_shm_pool *pool;   // ptr na deljeni pomnilnik
struct wl_buffer *buffer;       // ptr na del v pool-u
static int ix=100, iy=100, i=0;
static double transp=1.0;       // transparentnost
int fd, size;          // deskriptor datoteke, velikost v bajtih
int width = 500;
int height = 500;
int stride;
int ptrNum=0;
unsigned char *data;           // začetek podatkov v shm
cairo_t *cr;                  // cairo risalna površina
cairo_format_t cairo_format =
       CAIRO_FORMAT_ARGB32;  // format za izris
```

```
void draw_text(struct client_state *state){
  cr=cairo_create(state->csurface);//cairo_reference
  cairo_set_source_rgba(cr,1,0,1,transp);
  cairo_move_to(cr, ix, iy);// Lobster Two, Ani,
  cairo_select_font_face(cr,
            "Linux Biolinum Keyboard O",
            CAIRO_FONT_SLANT_NORMAL,
            CAIRO_FONT_WEIGHT_NORMAL);
  cairo_set_font_size(cr, 15);  // izbere se velikost pisave
  cairo_show_text(cr, "A Howdy!"); // prikaže se besedilo
  cairo_surface_flush(state->csurface);
   // izvede vse zahtevek in nariše vsebino na površino
  cairo_destroy(cr);           // uniči kairo površino
  wl_surface_attach(state->surface, buffer, 0, 0);
       // nastavi vsebino buferja kot vsebino te površine
  wl_surface_damage_buffer(state->surface, 0, 0,
                                  width, height);
                  // označi del površine za ponoven izris
  wl_surface_commit(state->surface);// prikaže vsebino
}
```

Wayland sistem → primeri

```
void draw_path(struct client_state *state){
  cr=cairo_create(state->csurface);//cairo_reference
  cairo_set_source_rgba(cr,1.0,0.2,0.2,1.0);
  cairo_move_to(cr, ix, iy);

  for (int i=0;i<360;i++){
    int dy=(int)(iy-sin(i*M_PI/180)*50);
    cairo_line_to(cr, ix+i, dy);
  }
  cairo_stroke(cr);

  cairo_surface_flush(state->csurface);
  cairo_destroy(cr);
  wl_surface_attach(state->surface, buffer, 0, 0);
  wl_surface_damage_buffer(state->surface, 0, 0,
                           width, height);
  wl_surface_commit(state->surface);
}
```

Zunanje knjižnice za risanje

```
static void keyboard_handle_keymap(void *data, struct wl_keyboard
    *keyboard, uint32_t format, int fd, uint32_t size){
  struct client_state *state = data;
  char *keymap_string = mmap (NULL, size, PROT_READ,
                              MAP_SHARED, fd, 0);
  xkb_keymap_unref (state->keymap);
  state->keymap = xkb_keymap_new_from_string (state->xkb_context,
          keymap_string,  XKB_KEYMAP_FORMAT_TEXT_V1,
                          XKB_KEYMAP_COMPILE_NO_FLAGS);
  munmap (keymap_string, size);
  close (fd);
  xkb_state_unref (state->xkb_state);
  state->xkb_state = xkb_state_new (state->keymap);
}
static void keyboard_handle_enter(void *data, struct wl_keyboard
  *keyboard, uint32_t serial, struct wl_surface *surface,
                              struct wl_array *keys){}

static void keyboard_handle_leave(void *data, struct wl_keyboard
    *keyboard, uint32_t serial, struct wl_surface *surface){}
```

Rokovalniki dogodkov

Wayland sistem → primeri

```
static void keyboard_handle_key(void *data, struct  wl_keyboard
  *keyboard, uint32_t serial, uint32_t time, uint32_t key,
    uint32_t bstate){
  struct client_state *state = data;
  if (bstate == WL_KEYBOARD_KEY_STATE_PRESSED){
    xkb_keysym_t keysym =
          xkb_state_key_get_one_sym (state->xkb_state, key+8);
    char name[64];
    xkb_keysym_get_name (keysym, name, 64);
    if (strcmp(name, "q")==0) exit (0);
    else if (strcmp(name,"KP_Add")==0){
      xdg_surface_set_window_geometry(state->xdg_surface, 0, 0,
                                          200, 200);
      wl_surface_commit(state->surface);
    } } }
static void keyboard_handle_modifiers(void *data, struct
  wl_keyboard *keyboard, uint32_t serial, uint32_t
  mods_depressed, uint32_t mods_latched, uint32_t
  mods_locked,uint32_t group){}
```

Rokovalniki dogodkov

```
static void handle_ping(void *data, struct xdg_wm_base *wm_base,
     uint32_t serial) { xdg_wm_base_pong(wm_base, serial);}
static void pointer_handle_leave(void *data, struct wl_pointer *pointer,
     uint32_t serial, struct wl_surface *surface){ }
static void pointer_handle_enter(void *data, struct wl_pointer *pointer,
     uint32_t serial, struct wl_surface *surface, wl_fixed_t sx, wl_fixed_t sy){
  struct client_state *state = data;
  struct wl_cursor_image *image;
  struct wl_buffer *buffer;
  state->ptr_default = wl_cursor_theme_get_cursor(
     state->ptr_theme, ptr_names[ptrNum]);
  ptrNum=(ptrNum+1)%totPtrs;
  if (state->ptr_default==NULL)   state->ptr_default =
        wl_cursor_theme_get_cursor(state->ptr_theme, "default");
  image = state->ptr_default->images[0];
  buffer = wl_cursor_image_get_buffer(image);
  struct wl_cursor_theme *theme;    theme=state->ptr_theme;
  wl_pointer_set_cursor(pointer, serial,state->ptr_surface,
                  image->hotspot_x, image->hotspot_y);
  wl_surface_attach(state->ptr_surface, buffer, 0, 0);
  wl_surface_damage(state->ptr_surface, 0, 0, image->width, image->height);
  wl_surface_commit(state->ptr_surface);
  ix=wl_fixed_to_int(sx); iy=wl_fixed_to_int(sy); }
```

Wayland sistem → primeri

```
static void pointer_handle_motion(void *data, struct wl_pointer
      *pointer, uint32_t time, wl_fixed_t sx, wl_fixed_t sy){
  ix=wl_fixed_to_int(sx);   iy=wl_fixed_to_int(sy);
}

static void pointer_handle_button(void *data, struct wl_pointer
      *ptr, uint32_t serial, uint32_t time, uint32_t button,
      uint32_t bstate){
  struct client_state *state = data;
  if (button == BTN_MIDDLE && bstate == 1)
    xdg_toplevel_move(state->xdg_toplevel, state->seat, serial);
  else if (button == BTN_LEFT && bstate == 1)
    draw_text(state);
  else if (button == BTN_RIGHT && bstate == 1)
    draw_path(state);
}
static void pointer_handle_axis(void *data, struct wl_pointer
      *wl_pointer, uint32_t time, uint32_t axis, wl_fixed_t value){
  transp=transp-0.01;
  if (transp<=0) transp=1.0;
}
```

Rokovalniki dogodkov

```
// Interface za poslušalca za miško
static const struct wl_pointer_listener pointer_listener = {
  .enter=pointer_handle_enter,
  .leave=pointer_handle_leave,
  .motion=pointer_handle_motion,
  .button=pointer_handle_button,
  .axis=pointer_handle_axis,
};

// Interface za poslušalca za vhodne naprave
static const struct wl_seat_listener seat_listener = {
  seat_handle_capabilities, // lahko tudi klicanje te metode iz main
};

// Interface za poslušalca za okno
const struct xdg_toplevel_listener xdg_toplevel_listener = {
  .configure = xdg_toplevel_configure_handler,
  .close = xdg_toplevel_close_handler
};
```

Wayland sistem → primeri

```
// Interface za poslušalca za tipkovnico
static const struct wl_keyboard_listener keyboard_listener = {
  keyboard_handle_keymap,
  keyboard_handle_enter,
  keyboard_handle_leave,
  keyboard_handle_key,
  keyboard_handle_modifiers,
};

// Interface za poslušalca za površino
const struct xdg_surface_listener xdg_surface_listener = {
  .configure = xdg_surface_configure_handler
};

// Interface za poslušalca za register objektov
const struct wl_registry_listener registry_listener = {
  .global = registry_global_handler,
  .global_remove = registry_global_remove_handler
};
```

Registracija rokovalnikov dogodkov

```
static void seat_handle_capabilities(void *data, struct wl_seat *seat,
                                     enum wl_seat_capability caps){
  struct client_state *state = data;
  if ((caps & WL_SEAT_CAPABILITY_POINTER) &&
                    !state->pointer) { // Miška
    state->pointer = wl_seat_get_pointer(seat);
    wl_pointer_add_listener(state->pointer, &pointer_listener, state);
  } else if (!(caps & WL_SEAT_CAPABILITY_POINTER) &&
                     state->pointer) {
    wl_pointer_destroy(state->pointer);
    state->pointer = NULL;
  }
  if (caps & WL_SEAT_CAPABILITY_KEYBOARD) { // Tipkovnica
    state->keyboard = wl_seat_get_keyboard(seat);
    wl_keyboard_add_listener(state->keyboard,
                      &keyboard_listener, state);
  } else if (!(caps & WL_SEAT_CAPABILITY_KEYBOARD)) {
    wl_keyboard_destroy(state->keyboard);
    state->keyboard = NULL;
  }
}
```

Registracija zmožnosti za sedež

Wayland sistem → primeri

```
// CB za konfiguriranje okna, trenutno se ne uporablja
void xdg_toplevel_configure_handler(void *data,
    struct xdg_toplevel *xdg_toplevel, int32_t width, int32_t height,
    struct wl_array *states) {
    printf("configure: %dx%d\n", width, height);
}

// CB ob zaprtju okna, trenutno se ne uporablja
void xdg_toplevel_close_handler(void *data,
    struct xdg_toplevel *xdg_toplevel) { printf("close\n");}


// CB za konfiguracijo površine
void xdg_surface_configure_handler(void *data,
    struct xdg_surface *xdg_surface,  uint32_t serial) {
  xdg_surface_ack_configure(xdg_surface, serial);
}
```

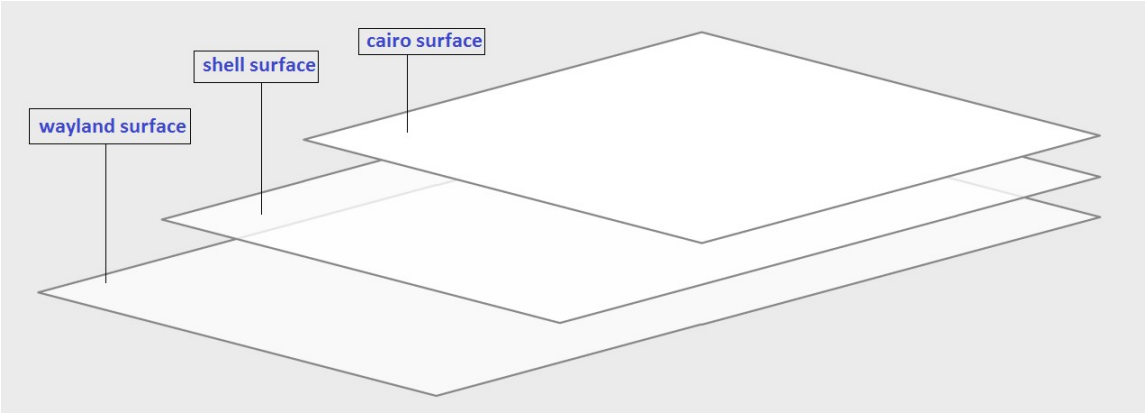
Registracija globalnih rokovalnikov

```
// CB za odstranitev globalnih objektov
void registry_global_remove_handler(void *data,
struct wl_registry *registry, uint32_t name) {}
```

```
// Registracija globalnih dogodkov, za katere se zanimamo
void registry_global_handler(void *data, struct wl_registry *rgsty,
uint32_t name, const char *interface, uint32_t version) {
  struct client_state *state = data;
  printf("Got a registry event for %s id %d\n", interface, name);
  if (strcmp(interface, "wl_compositor") == 0) { // Kompozitor
    state->compositor = wl_registry_bind(rgstry, name,
              &compositor_interface, 4);
  } else if (strcmp(interface, "wl_shm") == 0) { // SHM
       state->shm = wl_registry_bind(registry, name, &wl_shm_interface, 1);
       state->ptr_theme = wl_cursor_theme_load("Adwaita", 48, state->shm);
    // /usr/share/icons/.theme.
    state->ptr_default =
      wl_cursor_theme_get_cursor(state->ptr_theme, ptr_names[ptrNum]);
  } else if (strcmp(interface, "wl_seat") == 0) { // Seat
    state->seat = wl_registry_bind(registry, name, &wl_seat_interface, 1);
    wl_seat_add_listener(state->seat, &seat_listener, state);
  } else if (strcmp(interface, "xdg_wm_base") == 0) { // XDG_Shell
    state->xdg_wm_base =
      wl_registry_bind(registry, name, &xdg_wm_base_interface, 1);
  }
}
```

Wayland sistem → primeri

```
// Inicializacija risalne površine
void draw_cairo(struct client_state *state){
  static int i=10;
  void *myBuffer;
  cr=cairo_create(state->csurface);
  cairo_set_source_rgba(cr,0.95,1,1,0.85);
  cairo_rectangle (cr, 0, 0, width, height);
  cairo_fill(cr);
  cairo_move_to(cr, 0, height-i);
  cairo_set_source_rgba(cr,0,0,0.8,1);
  cairo_show_text(cr,"Initializing surface");
  cairo_surface_flush(state->csurface);
  myBuffer = cairo_image_surface_get_data(state->csurface);
  cairo_destroy(cr);
  wl_surface_attach(state->surface, buffer, 0, 0);
  wl_surface_damage_buffer(state->surface, 0, 0, width, height);
  wl_surface_commit(state->surface);
  i=i-10;
}
```



Wayland sistem → primeri

```
void setupWindow(struct client_state*state){
  state->display = wl_display_connect(NULL); // povežemo z vtičnikom ki ga odpre strežnik
  struct wl_registry *registry = wl_display_get_registry(state->display);
            // vrne objekt, ki omogoča poslušanje in povezovanje z globalnimi objekti
  wl_registry_add_listener(registry, &registry_listener, state); // dodamo poslušalca za register
  wl_display_roundtrip(state->display); // blokira izvajanje, dokler niso sprocesirane zahteve

  state->xkb_context = xkb_context_new (XKB_CONTEXT_NO_FLAGS); // kontekst tipkovnice

  state->surface = wl_compositor_create_surface(state->compositor);        // površina za risanje
  state->ptr_surface = wl_compositor_create_surface(state->compositor);  // površina za kurzor
  state->xdg_surface = xdg_wm_base_get_xdg_surface(state->xdg_wm_base, state->surface);
  // za površino pridobimo xdg površino, xdg_shell se pridobi v registry!!!
  xdg_surface_add_listener(state->xdg_surface, &xdg_surface_listener, NULL);
  // dodamo poslušalca za lupino (ni potrebno), ki posluša za pinge (vse dogodke)
  xdg_wm_base_add_listener(state->xdg_wm_base, &xdg_wm_base_listener, NULL);
  // dodamo poslušalca za xdg površino, potrebno, če želimo konfigurirati površino
  state->xdg_toplevel = xdg_surface_get_toplevel(state->xdg_surface);   // dodamo okno nad površino (lahko bi tudi popup)
  xdg_toplevel_add_listener(state->xdg_toplevel, &xdg_toplevel_listener, NULL);
  // dodamo še poslušalca za okno, potrebujemo za konfiguracijo ali zaprtje
  wl_surface_commit(state->surface); // izrišemo površino
```

Kreiranje osnovnih struktur in poslušalcev

Wayland sistem → primeri

```
...
  stride = width * 4;      // razlika med posameznimi piksli za risanje, ARGB -> 4 byte
  size = stride * height;  // bytes
  fd = syscall(SYS_memfd_create, "buffer", 0);   // odpri anonimno datoteko v RAMu in zapisi nekaj nicel
  ftruncate(fd, size * 2);
  // mapiraj v pomnilnik, data kaze na zacetek te pomniliske lokacije
  data = mmap(NULL, size, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
  // kreiraj povrsino za cairo
  state->csurface = cairo_image_surface_create_for_data (data, cairo_format, width, height, stride);   // pretvori v deljeni pomnilnik
  pool = wl_shm_create_pool(state->shm, fd, size * 2 );
  //alociraj bufer v pomnilniku, trenutno ne osvezuje, ce je ARGB8888
  buffer=wl_shm_pool_create_buffer(pool, 0, width, height, stride,WL_SHM_FORMAT_ARGB8888);
  wl_shm_pool_destroy(pool);
}
int main(void){
  setupWindow(&state);
  wl_display_roundtrip(state.display); // pocakaj, da se vsi zahtevki izvedejo
  draw_cairo(&state);                  // izrisemo in prikazemo
  while (1) { // Zanka dogodkov
    wl_display_dispatch(state.display);
  }
}
```

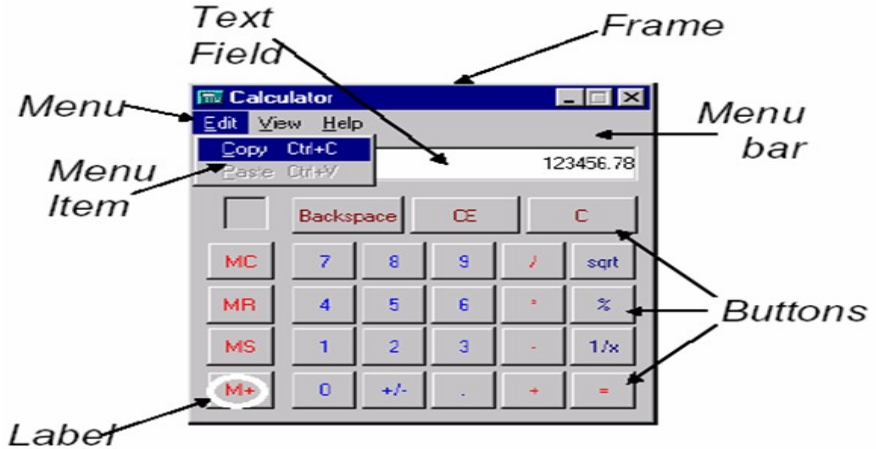
Kreiranje površine za risanje

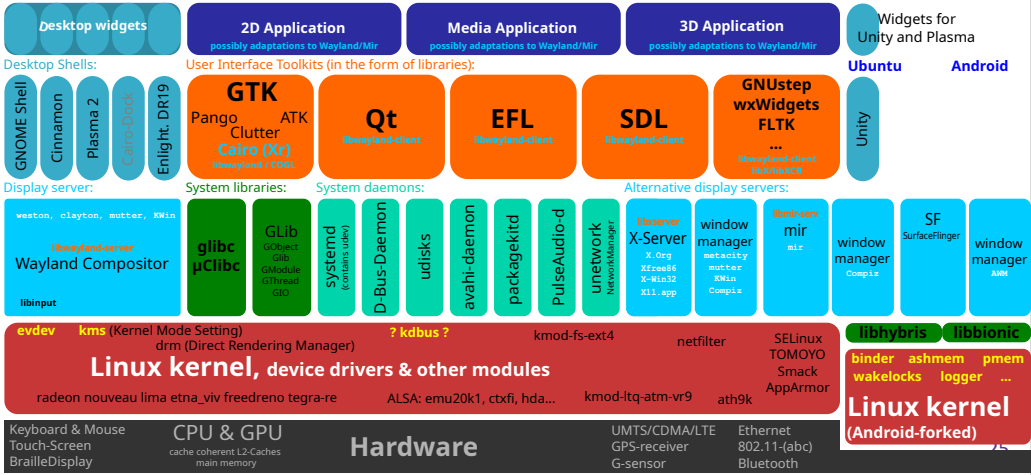
Zanka dogodkov

Orodja za načrtovanje uporabniških vmesnikov

UL FRI

- na voljo so knjižnice razredov, ki predstavljajo predloge za ustrezne grafične elemente
- objektom teh razredov (podobe) lahko nastavljamo različne lastnosti (barva, besedilo, velikost, položaj, …)
- nekatere podobe (vsebovalniki) lahko vsebujejo tudi eno ali več podob





Dogodkovni interaktivni vmesniki z grafičnim upor. vmes.

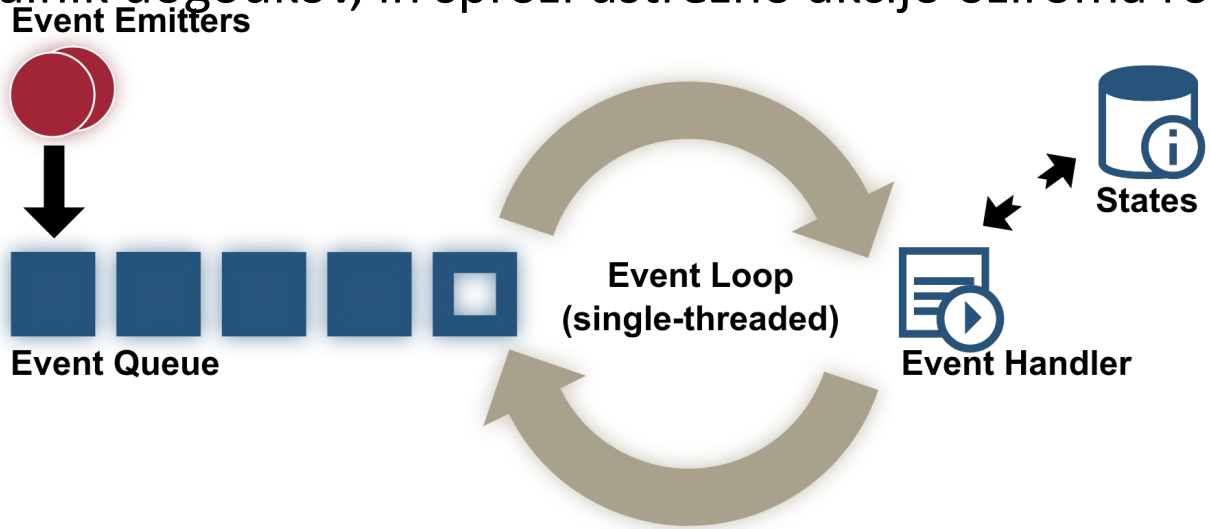
UL FRI

- Java – Swing/AWT/JavaFX
  - zanka dogodkov se izvaja avtomatično v ločeni niti
  - za zaznavo dogodkov moramo implementirati poslušalce:
    - ActionListener: *actionPerformed*
    - WindowListener: *windowClosed*
    - MouseListener: *mouseClicked*
    - MouseMotionListener: *mouseMoved*
    - .....

```
public Class implements ActionListener{
  public Class(){
    deklaracija;
    inicializacija;
    gradnja objektov GUV;
    registracija odzivnih funkcij;
    ...
  }
  public void actionPerformed(ActionEvent e)
    {...}
  public void listener1(){
     koda1;
  }
  public void listener2(){
     koda2;
  }
}
```

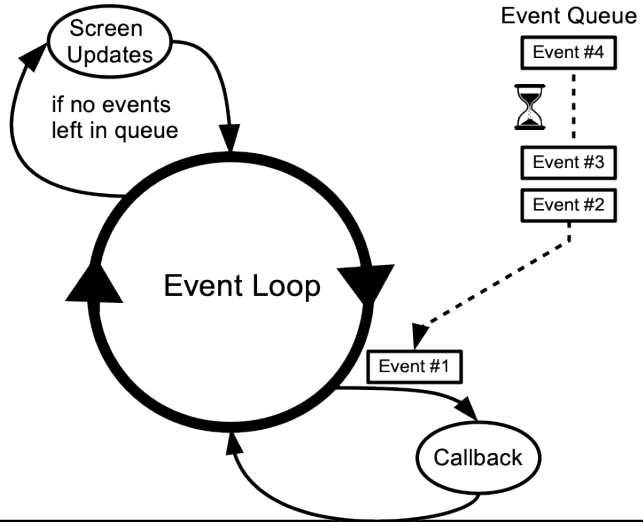
Obravnava dogodkov

- dogodki so objekti, ki so kreirani kot odziv na neko akcijo ali spremembo ….
- do dogodkov tipično pride ob interakciji uporabnika z neko podobo v GUV
- zanka dogodkov zazna dogodek, pogleda če je registriran ustrezen rokovalnik dogodkov, in sproži ustrezno akcijo oziroma rokovalnik dogodkov



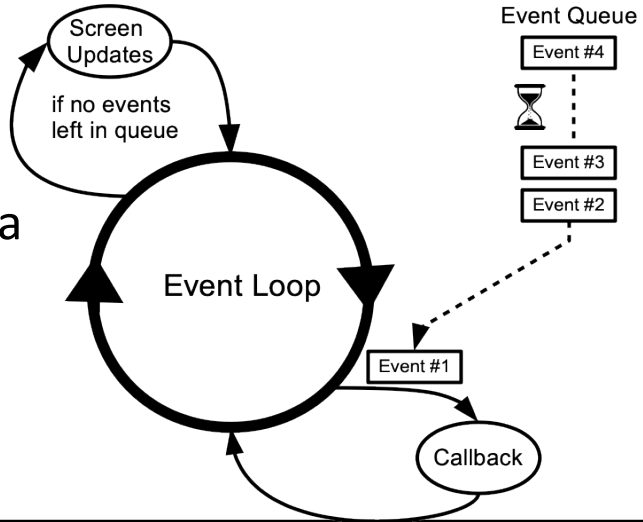
Vrsta dogodkov (*event queue*)

- dogodki se hranijo v vrsti (queue)
- rokovalnik dogodkov lahko porabi več časa za procesiranje dogodka, kot pa je čas med pojavitvijo dveh dogodkov
- pojavitev novega dogodka pred koncem obravnave prejšnjega lahko privede do težav (dostop do podatkov)
- vrsta prepreči pojavitev težav – dogodek se ne procesira, dokler prejšnji ni procesiran
- premiki miške se pretvorijo v en sam dogodek



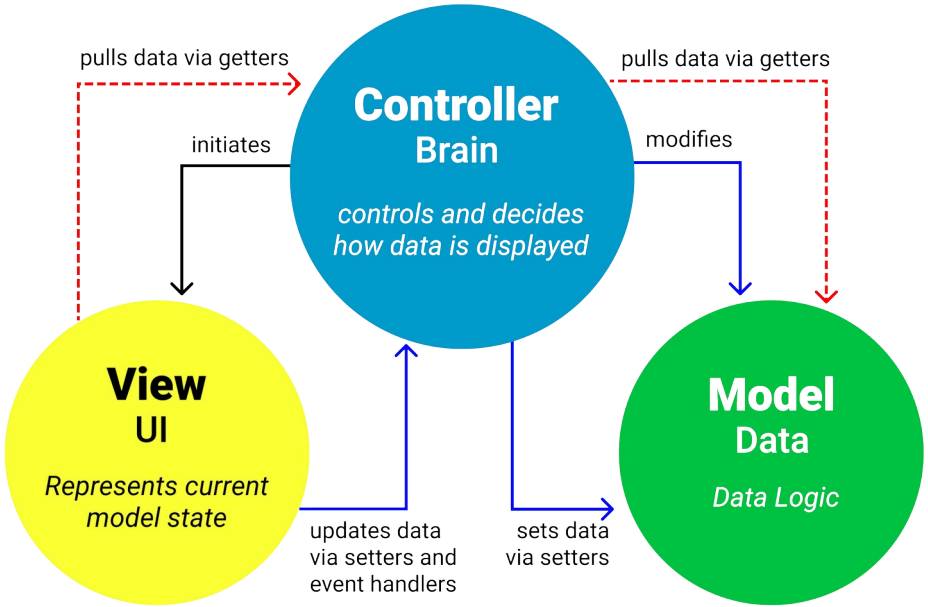
Zanka dogodkov

- čakanje na dogodek, dokler ni pripravljen
  - ko je pripravljen se vzame iz vrste
  - včasih se preprosti dogodki (pritisk/spust gumba miške, …) pretvorijo v preveden dogodek (klik ali dvojni klik gumba miške, fokus, znaki, …)
- zagotavljanje zanke dogodkov
  - napredna orodja za gradnjo GUV naredijo to interno
  - pri preprostih orodjih mora aplikacija to narediti sama



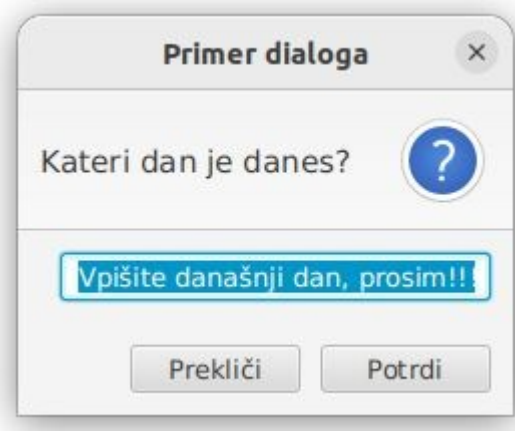
Model-pogled-nadzornik

- model (»Model« → M)
  - podatki povezani z aplikacijo/komponento
  - metode za dostop in spreminjanje podatkov in stanja
  - ni povezan z vmesnikom oziroma predstavitvijo
- pogled (»View« → V)
  - prikaže vsebino modela uporabniku v ustreznem vmesniku
  - omogoči uporabniku upravljanje s podatki
  - ob spremembi modela se mora pogled prilagoditi
- nadzornik (»Controller« → C)
  - posrednik med modelom in pogledom
  - prevede uporabnikove akcije v operacije na modelu
  - posodobi model ko uporabnik upravlja s pogledom



Motivacija za pristop MVC

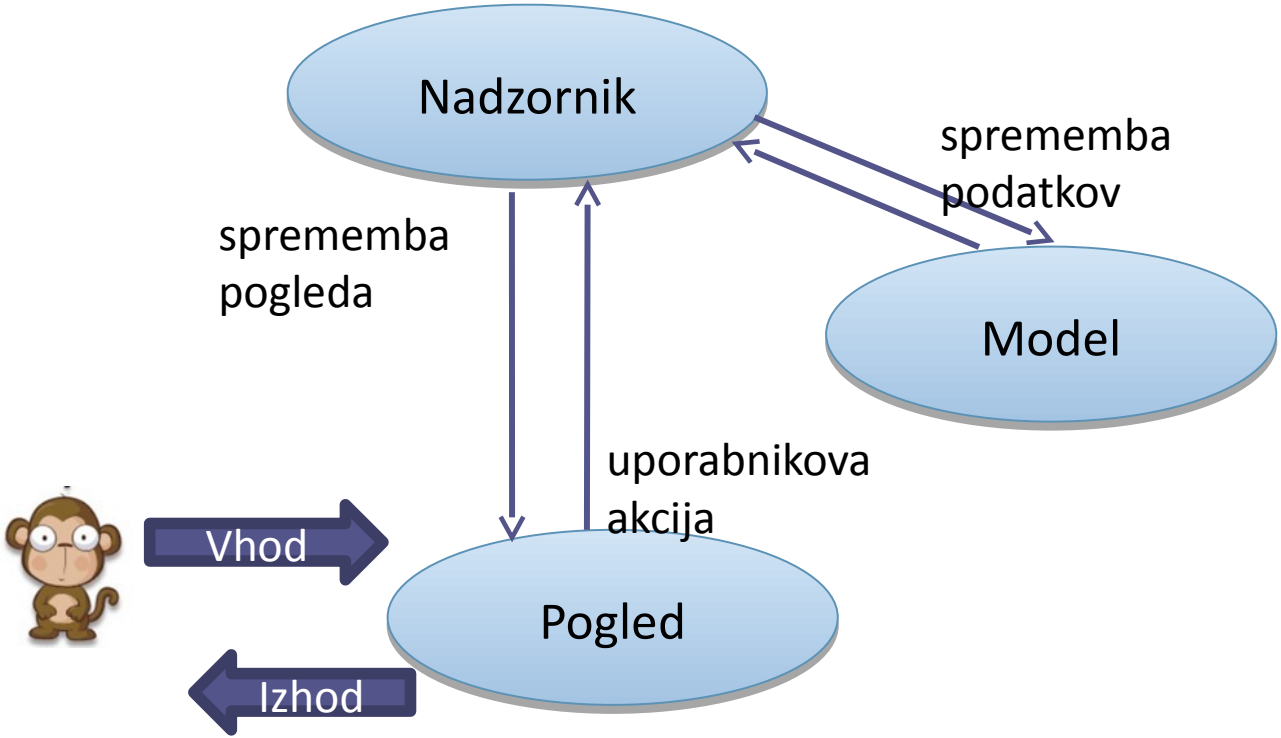
- osnovne komponente vsake interaktivne aplikacije/komponente:
  - podatki s katerimi se rokuje
  - uporabniški vmesnik, ki služi za rokovanje s podatki
- podatki so logično neodvisni od načina predstavitve uporabniku
  - prikaz naj bi bilo mogoče načrtovati ločeno
- primer takega pristopa: porazdelitev ocen pri predmetu
  - mogoča je predstavitev s stolpčnim in/ali tortnim diagramom
  - mogoča je predstavitev s črtami in/ali škatlami
- omogoča gradnjo generičnih podob

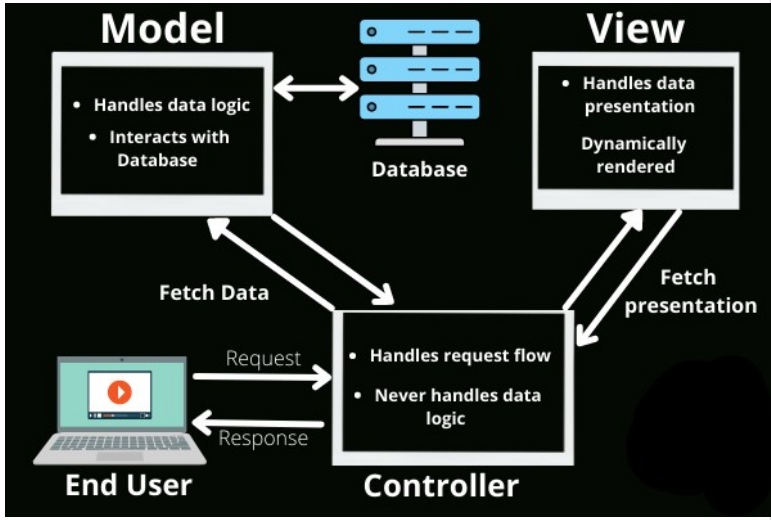


Zakaj uporaba MVC

- dobra ločitev predstavitve, podatkov in poslovne logike
  - omogoča lažje vzdrževanje aplikacije/komponente
  - omogoča izogibanje enemu razredu/objektu/..., ki bo storil vse
- ločevanje delov aplikacije/komponente omogoča ponovno uporabo
  - z zmanjševanjem odvisnosti komponent se lažje vzame komponenta in uporabi v drugi aplikaciji/komponenti
- omogoča zmanjševanje kompleksnosti delov aplikacije/komponente
  - kompleksnost je porazdeljena med različne dele
  - vsak del zase je manj kompleksen
- poveča fleksibilnost
  - vsak del aplikacije/komponente se lažje prilagodi kakršnim koli spremembam
- omogoča ločen razvoj, testiranje in vzdrževanje vsakega izmed delov aplikacije/komponente

Osnovna interakcija v MVC

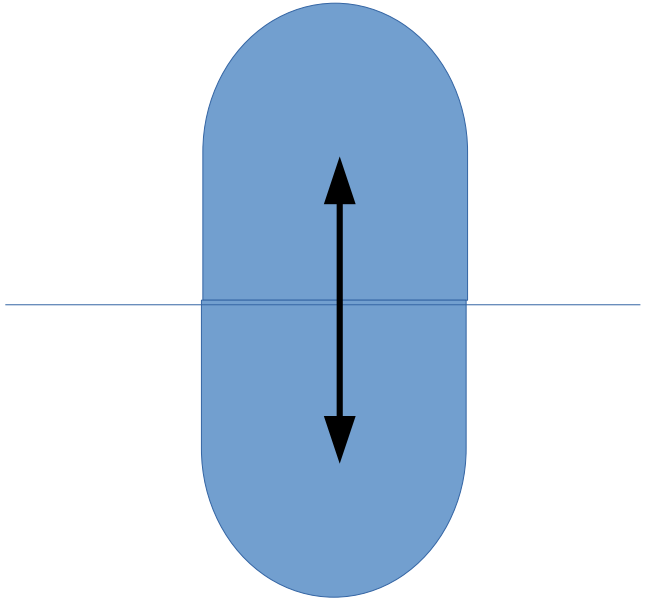




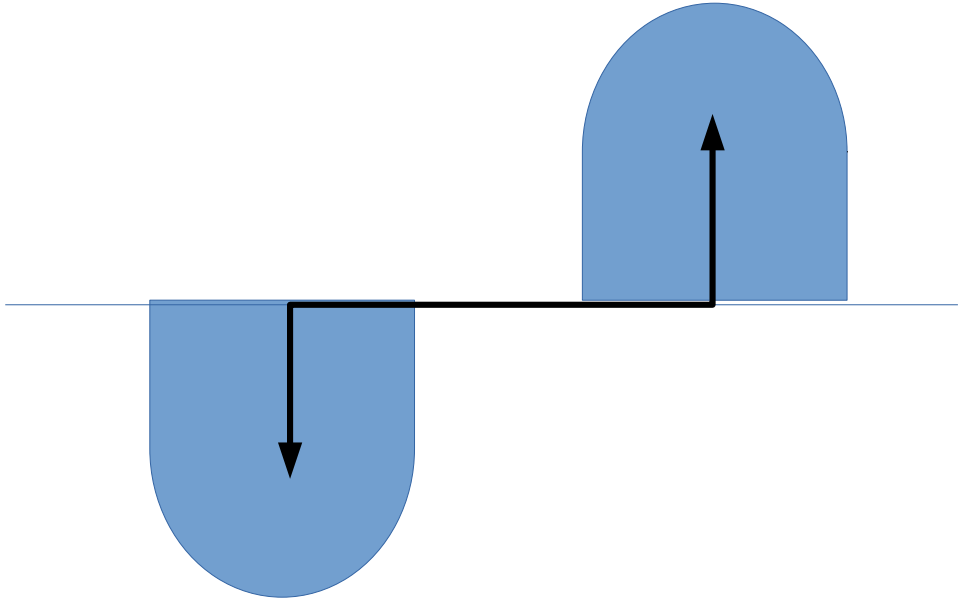
Osnovna interakcija v MVC

- uporabnik izvaja interakcijo z uporabniškim vmesnikom (pogled)
- nadzornik prejme vhod od uporabniškega vmesnika
- nadzornik spremeni model glede na uporabnikovo akcijo
- model procesira posodobitve, ki jih zahteva nadzornik, pogosto je to pisanje/branje v podatkovno bazo
- nadzornik posodobi ali spremeni pogled s posodobitvami modela
- posodobljen uporabniški vmesnik se prikaže in ponovno čaka na uporabnikovo interakcijo

Okenski sistem X (X11)

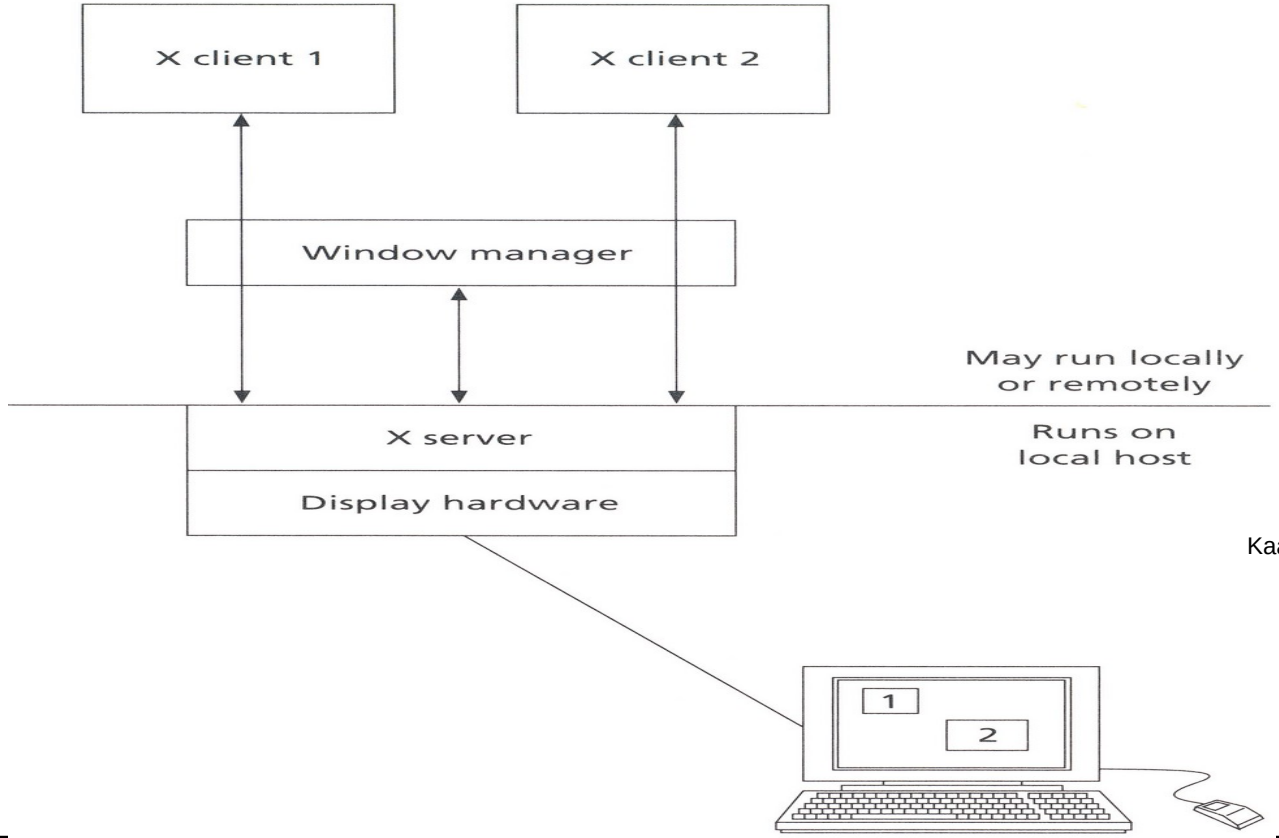


Strojna neodvisnost



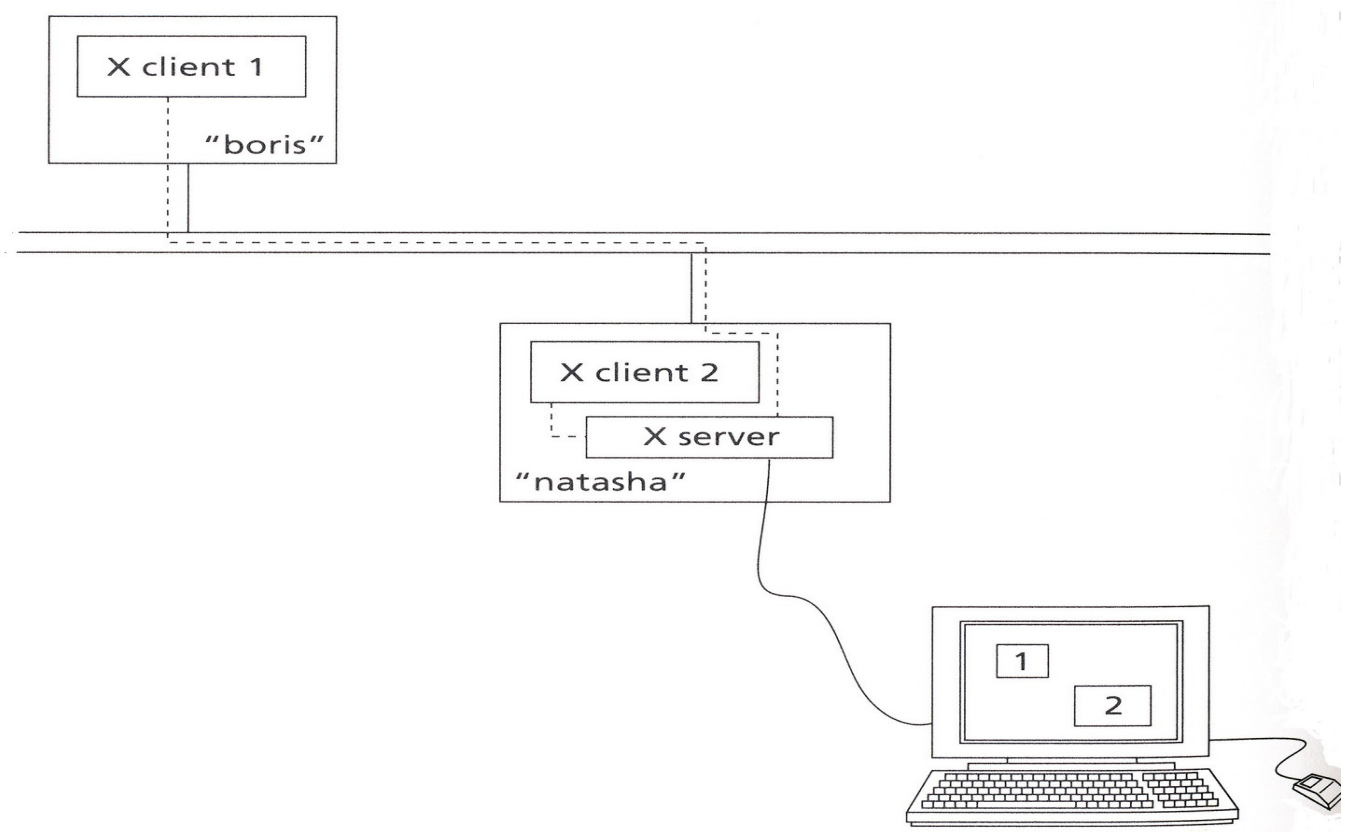
Pozicijska neodvisnost

Okenski sistem X (X11)



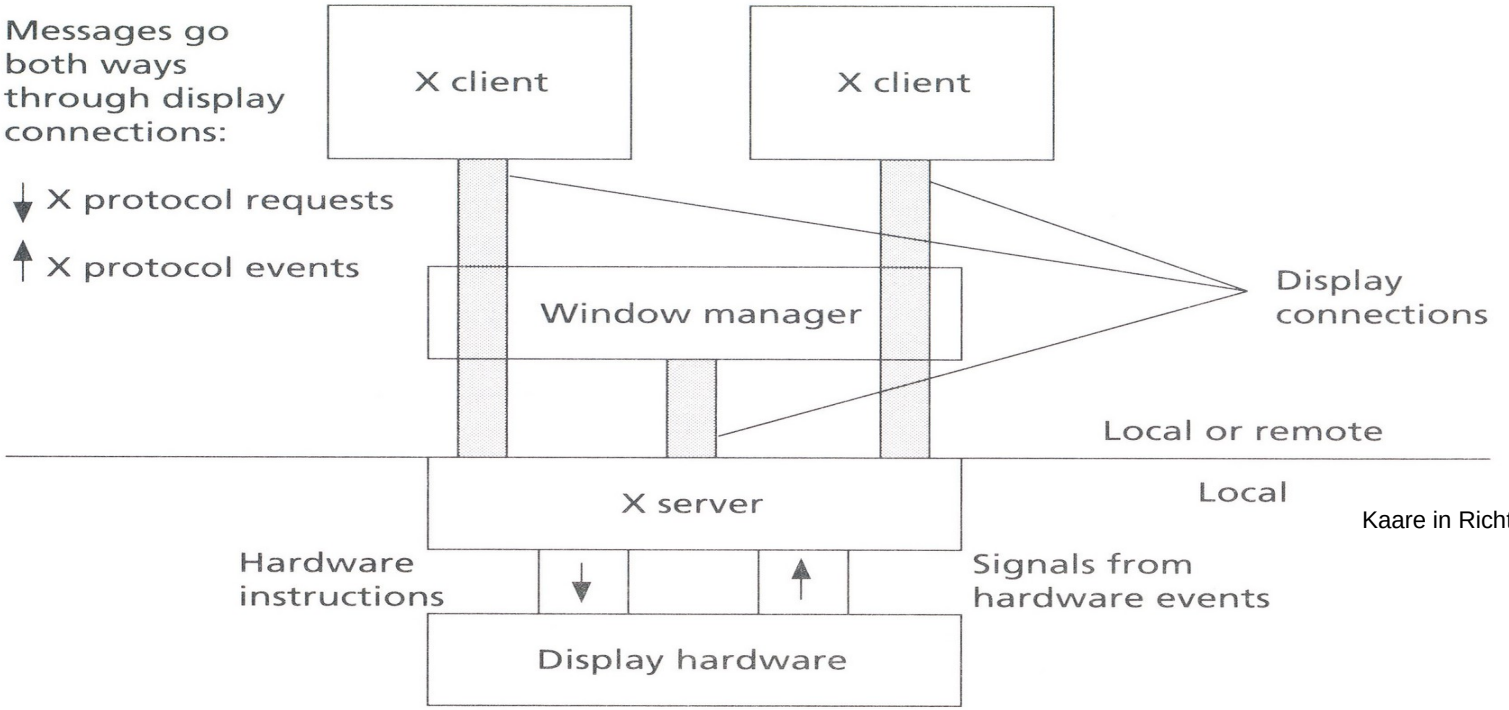
Kaare in Richter

Okenski sistem X (X11)



Kaare in Richter

Okenski sistem X (X11)



Kaare in Richter

Okenski sistem X (X11)

- prikazovalnik (DISPLAY) – abstrakcija, ki predstavlja vhodno/izhodne naprave (miška, tipkovnica, zaslon) uporabnika
- strežnik nadzoruje prikazovalnik (strežnik se nahaja na računalniku, za katerim sedi uporabnik)
  - nadzoruje dostop do prikazovalnika za odjemalce
  - pošilja sporočila po mreži
  - sledi zahtevam odjemalcev in posodablja okna
  - dvodimenzionalno risanje
  - sprejema vhodne dogodke in jih posreduje odjemalcem

Okenski sistem X (X11)

- odjemalci imajo dostop do prikazovalnika
  - izvajajo se sočasno
    - več oken istočasno
    - poganjanje aplikacij na več računalnikih istočasno
    - aplikacije so neodvisne
  - uporabljajo mrežni protokol preko vmesnika Xlib
  - med seboj komunicirajo preko strežnika
  - vsak odjemalec posreduje strežniku zahtevo za gradnjo vsaj enega okna, ki je naslednik osnovnega okna

Okenski sitem X (X11) → primer

- gradnja in izvajanje aplikacije v okenskem sistemu X
  - vzpostavitev povezave s strežnikom
  - nastavljanje lastnosti okna
  - napotki upravniku oken
  - gradnja grafičnega konteksta
  - prikaz okna
  - obravnavanje dogodkov v zanki dogodkov
  - razkroj okna
  - prekinitev povezave s strežnikom

Okenski sistem X (X11) → primer

UL FRI

```
#include <stdio.h>
#include <string.h>
#include <X11/Xlib.h>
#include <X11/Xutil.h>

char hello[] = {"Hello, World."};
char hi[]    = {"Hi!"};
int main(int argc, char **argv){
  Display  *mydisplay; /* Struktura nadzoruje povezavo s prikazovalnikom */
  Window   mywindow;   /* Okno, ki ga aplikacija zgradi */
  GC       mygc;       /* Graficni kontekst */
  XEvent   myevent;    /* Unija struktur za razlicne tipe dogodkov */
  KeySym   mykey;      /* Enolicno opise znak po "dogodku" s tipkovnice */
  XSizeHints myhint;   /* Tu proces posreduje informacijo o oknu upravniku oken */
  int      i, done,  myscreen;     /* stevilka zaslona */
  unsigned long mymask=0, myforeground, mybackground;
  char     text[10];
  XSetWindowAttributes  myattr;   /* Atributi okna */

/* Inicializacija */
mydisplay = XOpenDisplay(""); /* Inicializija povezave aplikacije in streznika X/

/* Kateri zaslon je v uporabi */
myscreen     = DefaultScreen(mydisplay);
mybackground = WhitePixel(mydisplay, myscreen);
myforeground = BlackPixel(mydisplay, myscreen);
               /* Barve za risanje ozadja tock, teksta in robov */
```

```
typedef struct {
  int type;
  unsigned long serial;
  Bool send_event;
  Display *display;
  Window window;
} XAnyEvent;
```

```
Pixmap background_pixmap;
unsigned long background_pixel;
Pixmap border_pixmap;
unsigned long border_pixel;
int bit_gravity;
int win_gravity;
int backing_store;
unsigned long backing_planes;
unsigned long backing_pixel;
Bool save_under;
long event_mask;
long do_not_propagate_mask;
Bool override_redirect;
Colormap colormap
Cursor cursor;
```

```
mydisplay = XOpenDisplay("");
```

```
myscreen     = DefaultScreen(mydisplay);
mybackground = WhitePixel(mydisplay, myscreen);
myforeground = BlackPixel(mydisplay, myscreen);
```

Okenski sistem X (X11) → primer

```
myhint.x = 200;     myhint.y = 300;
/* Sirina, dolzina */
myhint.width = 350; myhint.height = 250;
myhint.flags = PPosition | Psize;  /* Katere komponente v strukturi XSizeHints */


/* Zahteva postaji, da zgradi okno aplikacije */
mywindow = XCreateSimpleWindow(mydisplay,
            DefaultRootWindow(mydisplay), /* predhodnik */
            myhint.x, myhint.y, myhint.width, myhint.height,
            5, myforeground, mybackground);

/* Informacija o na novo zgrajenem oknu za upravnika oken */
XSetStandardProperties(mydisplay, mywindow,
                       hello, hello, None, argv, argc, &myhint);

mygc = XCreateGC(mydisplay, mywindow, 0, 0); /* Zgradi graficni kontekst */
XSetBackground(mydisplay, mygc, mybackground);
XSetForeground(mydisplay, mygc, myforeground);

/* Izberemo obvestila o vsakem dogodku z miske, tipkovnice, ali ob osvezevanju zaslona */
XSelectInput(mydisplay, mywindow,  ( ButtonPressMask | KeyPressMask | ExposureMask));

/* Prikaz okna na zaslon */
XMapRaised(mydisplay, mywindow);
```

```
myhint.flags = PPosition | PSize;
```

```
mywindow = XCreateSimpleWindow(mydisplay,
            DefaultRootWindow(mydisplay),
            myhint.x, myhint.y,
            myhint.width, myhint.height,
            5, myforeground, mybackground);
```

```
mygc = XCreateGC(mydisplay, mywindow, 0, 0);
```

```
XSelectInput(mydisplay, mywindow,
    (ButtonPressMask|KeyPressMask|ExposureMask);
```

```
XMapRaised(mydisplay, mywindow);
```

Okenski sistem X (X11) → primer

```
done = 0;
while (done == 0) {  /* Zanka dogodkov */
  XNextEvent (mydisplay, &myevent);  /* naslednji dogodek */
  switch (myevent.type) {
    case KeyPress:     /* dogodek s tipkovnice */
       i = XLookupString(&myevent.xkey, text, 10, &mykey, NULL);
       if (i == 1 && text[0] == 'q') done = 1;
       break;
    case Expose:       /* dogodek ob zagonu programa */
      if (myevent.xexpose.count == 0)
          XdrawImageString(myevent.xexpose.display, myevent.xexpose.window,
             mygc, 50, 50  /* pozicija */,
             hello, strlen(hello) );
       break;
    case ButtonPress:  /* dogodek z miske */
        XdrawImageString(myevent.xbutton.display, myevent.xbutton.window,
           mygc, myevent.xbutton.x, myevent.xbutton.y,
           hi, strlen(hi));
       break;
  } /* switch */
} /* while */
XFreeGC(mydisplay,mygc);/* Razkroji graficni kontekst */
XDestroyWindow(mydisplay, mywindow);/* Razkroji okno */
XCloseDisplay(mydisplay);/* Prekine povezavo s streznikom X */
return 0;
}
```

```
while (done == 0) { … }
```

```
case KeyPress:  {
  i=XLookupString(&myevent.xkey,
        text, 10, &mykey, NULL);
  …
}
```

```
case Expose: { … }
```

```
case ButtonPress: { … }
```

```
XFreeGC (mydisplay, mygc);
```

```
XDestroyWindow(mydisplay, mywindow);
```

```
XCloseDisplay(mydisplay);
```

Okenski sistem X (X11) → primer generiranja lastne podobe

```
Window MyCreateSimpleButton(Display *dspl, Window prt, GC mygc,
                int x, int y, int w, int h, char *txt){
  Window mw;
  Font font;

  mw  = XCreateSimpleWindow(dspl, prt, x, y, w, h, 3, 0x00ccde, 0xcccccc);
  gcv.foreground = 0xabcdef;
  gcv.background = 0x612345;
  gcv.font = XLoadFont (dspl, "9x15");
  XGrabButton(dspl, AnyButton, AnyModifier, mw, True,
              ButtonPressMask | ButtonReleaseMask,
              GrabModeAsync, GrabModeAsync, None, None);
  bttxt = (char *)malloc(sizeof(char)*strlen(txt)+1);
  sprintf(bttxt,"%s", txt);
  bth=h; btw=w;
  btgc = XCreateGC(dspl,RootWindow(dspl, DefaultScreen(dspl)) ,
                   GCFont | GCForeground | GCBackground , &gcv);
  XMapRaised(dspl, mw);

  XDrawImageString(dspl, mw, btgc, 22, (h+10)/2, txt, strlen(txt));
  return mw;
}
```

```
XCreateSimpleWindow(dspl, prt, x,y,w,h,3, 0x00ccde, 0xcccccc);
```

```
XGrabButton(dspl, AnyButton, AnyModifier, mw,
       True, ButtonPressMask | ButtonReleaseMask,
       GrabModeAsync, GrabModeAsync, None, None);
```

```
XMapRaised(dspl, mw);
```

```
XDrawImageString(dspl, mw, btgc, 22, (h+10)/2, txt, strlen(txt);
```

Okenski sistem X (X11) → primer generiranja lastne podobe

UL FRI

```
mybt = MyCreateSimpleButton(mydisplay, mywindow, mygc, 70, 70, 160, 60, "Neko besedilo");
```

```
MyCreateSimpleButton(mydisplay, mywindow, mygc, 70, 70, 160, 60, »Neko besedilo«
```

```
int ButtonPressed(Display *dspl, Window mw){
  XSetWindowBackground(dspl, mw, 0x040404);
  XClearWindow(dspl, mw);
  XSetBackground(dspl, btgc, 0xabcdef);
  XSetForeground(dspl, btgc, 0x612345);
  XDrawImageString(dspl, mw, btgc, 22, (bth+10)/2, bttxt, strlen(bttxt));
}
```

```
int ButtonPressed(Display *dspl, Window mw){
  …
}
```

```
int ButtonReleased(Display *dspl, Window mw){
  XSetWindowBackground(dspl, mw, 0xcccccc);
  XClearWindow(dspl, mw);
  XSetBackground(dspl, btgc, 0x612345);
  XSetForeground(dspl, btgc, 0xabcdef);
  XDrawImageString(dspl, mw, btgc, 22, (bth+10)/2, bttxt, strlen(bttxt));
}
```

```
int ButtonReleased(Display *dspl, Window mw){
  …
}
```